GRUPO 3

# Ciclo de Vida e Gestão de Estado

Garantir a persistência de dados e fluidez da UI num ecossistema Android em constante mutação.

# O que vamos abordar



## O Ciclo Base

Atividades, Fragmentos e a ameaça das alterações de configuração.



## Persistência

Bundles, onSaveInstanceState e o robusto ViewModel.



## Reatividade

A ponte para a UI através de LiveData e StateFlow.

# O Ciclo de Vida • Activity

Sequência de estados fundamentais e callbacks de transição da interface.

CRIAÇÃO

## `onCreate()`

Inicialização da UI e variáveis estáticas.
Executado uma única vez ao criar a Activity.

VISIBILIDADE

## `onStart()`

Activity torna-se visível ao utilizador.
Prepara a UI para entrar em primeiro plano.

INTERAÇÃO

## `onResume()`

Foco principal e interação ativa.
**Estado ativo: pronto para receber dados do utilizador.**

SEGUNDO PLANO

## `onPause()` / `onStop()`

Perda de foco e transição para segundo plano.
**Pausa operações visuais ou liberta dados temporários.**

DESTRUIÇÃO

## `onDestroy()`

Libertação final de recursos.
**Encerramento definitivo da instância da Activity.**

# O Ciclo de Vida • Fragment

Etapas essenciais de criação e destruição da interface desvinculada da Activity

## 1. Inicialização

Ligação com a Activity hospedeira

**`onAttach()`**
Fragment associado à Activity

↓

**`onCreate()`**
Inicialização de componentes

## 2. Interface Gráfica (UI)

Criação e interação com a View

**`onCreateView()`**
Gera a hierarquia de Views

↓

**`onViewCreated()`**
View pronta: setup de observers/flows

↓

**`onStart() / onResume()`**
**Fragment visível e ativo**

## 3. Destruição & Limpeza

Desvinculação de recursos

**`onDestroyView()`**
Limpeza de referências da View

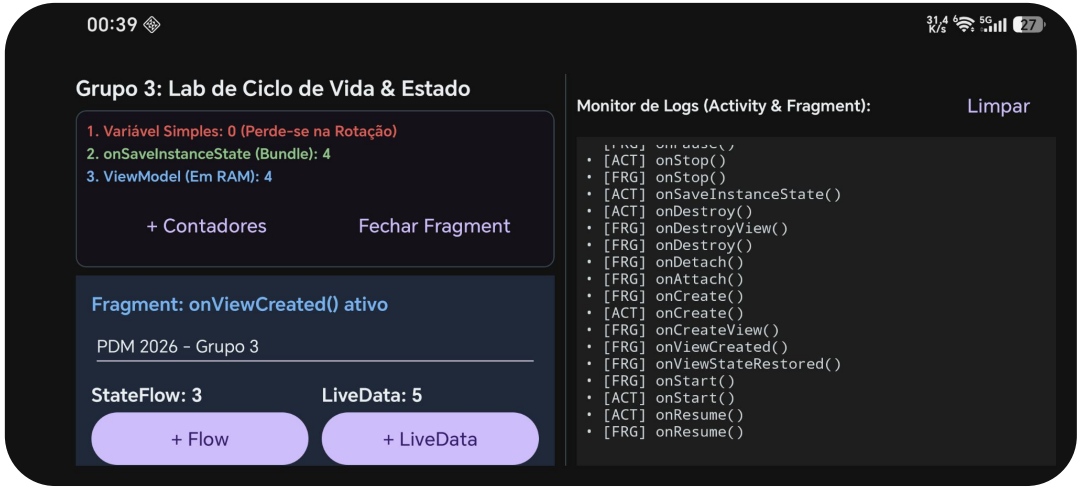
↓

**`onDestroy() / onDetach()`**
Destruição final do Fragment




# O problema da rotação de ecrã

- ❌ Activity é totalmente destruída.
- 🔄 Nova instância recriada do zero.
- ⚠️ Dados não guardados desaparecem.



# A Primeira Linha de Defesa

## onSaveInstanceState

Guarda dados primitivos num **Bundle**.

Executado antes da Activity ser destruída.

Limitado a pequenos volumes de dados.



**Java:**

```java
@Override
protected void onSaveInstanceState(@NonNull Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putInt("KEY_BUNDLE", bundleCounter);
}
```

**Kotlin:**

```kotlin
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putInt("KEY_BUNDLE", bundleCounter)
}
```

A SOLUÇÃO MODERNA

# ViewModel

Separa a lógica e os dados da Interface Gráfica. Sobrevive imaculado a qualquer alteração de configuração.

**0**

Perda de dados na rotação

**100%**

Independência da UI

# E se o sistema matar a app?

## SavedStateHandle

O ViewModel não sobrevive à morte do processo (System-initiated process death).

Integrado no ViewModel, este mapa chave-valor garante a restauração completa dos dados críticos após a ressurreição da app.



# Observar as Alterações

## LiveData

**Lifecycle-aware:** Respeita a UI.

**Origem:** Java/Android Architecture.

**Uso:** Simples e direto.

```
viewModel.getLiveDataCounter().observe(
  getViewLifecycleOwner(), count → {
   binding.tvLiveData.setText(String.valueOf(count));
  }
);
```

## StateFlow

**Coroutines:** Moderno e nativo (Kotlin).

**Operadores:** Rico em transformações.

**Padrão atual:** Recomendado pela Google.

```
viewLifecycleOwner.lifecycleScope.launch {
  viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
    viewModel.flowCounter.collect {
      count → binding.tvFlow.text = "$count"
    }
  }
}
```

# Limitações e Erros Frequentes

Memory Leaks

## Fugas de Memória no Fragment

O ciclo de vida da View é mais curto que o do Fragment. Se não anularmos o View Binding a interface gráfica inteira fica presa na memória RAM.

Coroutines

## Consumo Invisível de Bateria

Fazer `lifecycleScope.launch` num StateFlow sem `repeatOnLifecycle` mantém a coroutine a processar eventos em segundo plano mesmo com a app minimizada.

Bundle OS

## Limite Rígido de Memória

O Bundle de `onSaveInstanceState` partilha um buffer do SO de ~1 MB. Tentar guardar listas grandes ou imagens causa `TransactionTooLargeException` e fecha a app.

PERGUNTAS & DISCUSSÃO

# Questões para Análise

QUESTÃO 01

Se o ViewModel já sobrevive à rotação, quando é que precisamos mesmo do **SavedStateHandle**?

QUESTÃO 02

Qual é o risco de observar um StateFlow sem **repeatOnLifecycle**?